

Chapitre 7 Variables pointeurs

Les variables de type pointeur sont des variables qui contiennent des adresses physiques réelles de la mémoire de l'ordinateur. Les adresses sont des nombres codés en hexadécimal (voir le paragraphe Codage hexadécimal page 18).

Type pointeur

Une variable de type pointeur est une variable qui contient l'adresse d'une zone mémoire. La plupart des pointeurs en C++ sont typés, c'est à dire qu'ils représentent à la fois l'emplacement d'une zone mémoire et la taille de cet emplacement mémoire (zone de stockage d'un réel, d'un entier, d'un objet, d'une fonction, ...). Pour déclarer une variable de type pointeur, il suffit de faire précéder son identificateur du caractère `*`.

Syntaxe

```
type* pVariable = NULL;
```

où `type*` est, pour les pointeurs sur des types prédéfinis, choisi parmi les valeurs du tableau suivant :

	Type	Occupation mémoire
Pointeurs sur des entiers	<code>char*</code>	- La taille et l'occupation en mémoire sont les mêmes pour tous les pointeurs, puisqu'ils contiennent tous une adresse physique de la mémoire. - En C++ comme en C, la taille des pointeurs dépend de la taille du bus d'adresses des processeurs utilisés (comme la taille de l'entier <code>int</code> dépend de la taille du bus de données du microprocesseur). - Sur les ordinateurs actuels, la taille du bus d'adresses du microprocesseur, et donc la taille des pointeurs, est 8 octets. Avec Windows 64 bits, la mémoire adressable peut aller jusqu'à 128 Go.
	<code>unsigned char*</code>	
	<code>short*</code>	
	<code>unsigned short*</code>	
	<code>int*</code>	
	<code>unsigned int*</code>	
	<code>long*</code>	
	<code>unsigned long*</code>	
Pointeurs sur des réels	<code>float*</code>	
	<code>double*</code>	
	<code>long double*</code>	
	<code>void*</code>	

Tableau 13 : pointeurs sur les types prédéfinis du C++.

Remarques

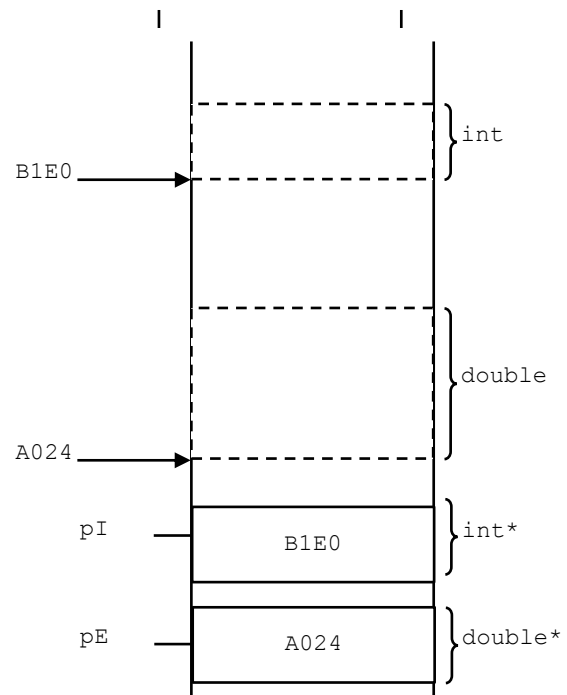
- La valeur d'une variable pointeur peut être soit la valeur réservée **NULL** (équivalent de la valeur 0 pour les pointeurs), soit l'adresse d'une variable (voir paragraphe « Opérateur & », page 42).
- Une erreur fréquente est d'omettre l'initialisation des variables pointeurs qui pointent alors n'importe où en mémoire (données, code, système d'exploitation, etc ...).
- Déclarer une variable pointeur revient à réserver un emplacement qui permet de stocker une adresse, mais pas l'emplacement pointé par cette adresse (voir opérateur new).
- Il est d'usage d'utiliser la lettre minuscule p devant l'identificateur de la variable pour rappeler qu'il s'agit d'une variable pointeur. Dans ce cas, on ne rappelle le type de la zone pointée.
- Les programmeurs C++ ont l'habitude d'accoler l'étoile et le type de la zone pointée. Inversement, les programmeurs C accolent l'étoile à l'identificateur de la variable.

Exemples

`int* pI = NULL;` définit une variable `pI` qui contient l'adresse d'un emplacement mémoire pouvant contenir un `int`

`double* pE = NULL;` définit un pointeur `pE` sur un `double`

`float** pA = NULL;` définit une variable `pA` qui pointe sur un pointeur qui pointe sur un `float`.



Pointeur void

Le type particulier pointeur `void` définit un pointeur qui pointe une zone mémoire sans spécifier le type de la zone pointée.

Syntaxe

```
void* pVariable = NULL;
```

Opérateur &

L'opérateur `&` est un opérateur unaire qui a comme seul opérande un identificateur de variable ou de fonction. Il retourne l'adresse de la variable ou de la fonction, qui peut ensuite être affectée à toute variable pointeur.

Syntaxe

```
&identificateurDeVariable;
```

Remarque

- On dit que `&` retourne l'adresse de la variable.



Ne pas confondre l'opérateur `&identificateurDeVariable` avec le et logique (`&&`), le et bit à bit (`&`), ni même avec la déclaration d'une variable de type référence (`type&`).

Exemple

```
int nI = 0;
int* pI = &i;
```

Le pointeur sur un entier `pI` contient l'adresse de la variable entière `nI`.

Opérateur *

L'opérateur `*` (appelé opérateur de déréférenciation) permet d'accéder au contenu de la zone mémoire dont l'adresse est dans un pointeur. Il suffit pour cela de faire précéder l'identificateur du pointeur par le symbole `*`.

Syntaxe

```
*pVariablePointeur
```

Remarque

- l'opérateur `*` permet de « passer » du pointeur à la variable pointée.

Exemple

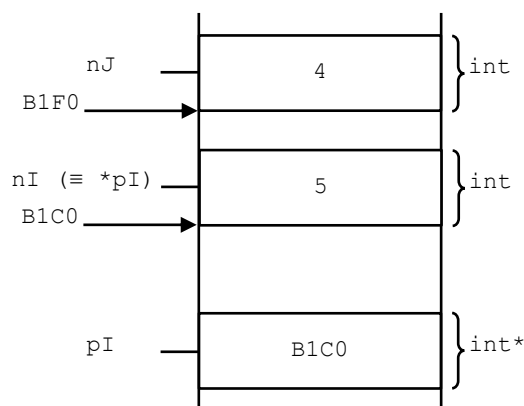
```
int* pI = NULL ;    // pI est initialisé par NULL
int nI = 4;         // nI est initialisée avec 4

pI = &nI;           // pI est initialisé avec l'adresse de nI.
                    // on dit "pI pointe sur nI"
cout << *pI;        // affiche le contenu pointé par pI,
                    // donc le contenu de la variable nI)

int nJ = *pi;        // nJ est initialisée avec la valeur de nI

*pI = 5;             // le contenu pointé par pI prend la valeur 5
                    // la variable nI contient donc maintenant 5
```

Représentation mémoire de l'exemple :



Pointeurs constants

Pour déclarer un pointeur constant, il faut, lors de la déclaration de ce pointeur, placer le mot réservé `const` entre le caractère `*` et son identificateur.

Exemple 1

```
int nI = 10;
int* const pI = &nI; // pI est un pointeur constant sur l'entier nI
```

De même, pour déclarer un pointeur sur une donnée constante, il faut placer le mot réservé `const` avant le caractère `*` et l'identificateur du pointeur.

Exemple 2

```
int nI = 10;
const int* pI = &nI; // pI est un pointeur sur l'entier constant nI
```

L'instruction `*pI = nI;` est autorisée dans le premier exemple (puisque le pointeur est constant mais pas la zone mémoire pointée), mais pas dans le second (la zone pointée est constante). Inversement, l'instruction `pI = &nI;` est correcte dans le second exemple (puisque le pointeur n'est pas constant) mais pas dans le premier (hormis lors de la déclaration du pointeur constant).

Opérations sur les pointeurs

Il est possible de réaliser des additions et des soustractions avec des variables de type pointeur. Par exemple `pI + 1` pointe l'emplacement mémoire qui suit celui pointé par `pI` (le décalage (en octets) réalisé dépend de la taille du type de `pI` : 2 pour `short`, 8 pour `double`, ...).